Vite & Gourmand

TRAITEUR ARTISANAL BORDELAIS

Documentation de gestion de projet

Méthodologie retenue, planification, priorisation MoSCoW, sprints Scrum, diagramme de GANTT, matrice RACI et retour d'expérience du projet Vite & Gourmand.

Version du document : 2.0 · **Date :** 23 juillet 2026

Auteur : Laurent MARC

Sommaire

1. Cadrage du projet 03

1.1 Contexte et vision produit 03

1.2 Objectifs SMART 04

1.3 Parties prenantes 04

2. Méthodologie retenue 05

2.1 Choix d'un modèle hybride 05

2.2 Cadre Scrum adapté 06

2.3 Outils de gestion 07

3. Priorisation MoSCoW 08

3.1 Must Have 08

3.2 Should Have 09

3.3 Could Have 10

3.4 Won't Have 10

4. Planification 11

4.1 Découpage en cinq sprints 11

4.2 Diagramme de GANTT 13

4.3 Chemin critique 14

5. Suivi opérationnel 15

5.1 Tableau Trello 15

5.2 Rythme et cérémonies 16

5.3 Matrice RACI 17

6. Gestion des risques 18

6.1 Identification des risques 18

6.2 Plans de mitigation 19

7. Bilan et retour d'expérience 20

7.1 Ce qui a bien fonctionné 20

7.2 Difficultés rencontrées 21

7.3 Enseignements pour la suite 22

1. Cadrage du projet

Contexte, vision produit, objectifs mesurables et cartographie des parties prenantes.

1.1 Contexte et vision produit

Le projet Vite & Gourmand est né d'un besoin fictif imaginé pour l'Épreuve de Certification Finale du titre professionnel Développeur Web et Web Mobile de Studi. Julie et José, jeunes entrepreneurs bordelais de 25 ans, souhaitent lancer leur activité de traiteur artisanal et cherchent une plateforme web capable de porter à la fois leur vitrine commerciale et leurs opérations quotidiennes.

Le besoin exprimé se structure en deux volets complémentaires :

- **Une vitrine numérique** pour présenter la marque, mettre en avant les menus par thème et par régime, et faciliter la prise de contact des prospects
- **Un outil de gestion intégré** pour éviter le recours à des tableurs et boîtes mails éparses lors du suivi des commandes, de la coordination des équipes et du pilotage de l'activité

Trois segments de clientèle sont ciblés dès l'origine : particuliers pour les événements familiaux (mariages, anniversaires), entreprises pour les cocktails et séminaires, associations pour les manifestations culturelles. Cette diversité influe directement sur la conception de la plateforme, qui doit rester lisible pour un client occasionnel tout en offrant la richesse fonctionnelle qu'attendent des acheteurs professionnels.

> **Vision produit synthétique**
>
> Offrir aux traiteurs artisanaux une plateforme complète leur permettant de se digitaliser sans dépendre de solutions clé en main coûteuses, tout en préservant leur identité de marque et la conformité réglementaire française (RGPD, LCEN, RGAA).

1.2 Objectifs SMART

Les objectifs du projet ont été formulés selon la méthode SMART pour garantir leur clarté et leur mesurabilité.

Critère	Formulation
Spécifique	Développer une plateforme web full-stack couvrant les parcours visiteur, client, employé et administrateur d'un traiteur bordelais
Mesurable	51 endpoints REST fonctionnels, 17 tables PostgreSQL, 5 documents livrables PDF, un site accessible en HTTPS avec certificat valide
Atteignable	Périmètre défini via MoSCoW pour ajuster l'effort en fonction du temps disponible, avec identification des Won't Have dès le cadrage
Réaliste	Basé sur les compétences DWWM enseignées (Symfony, PostgreSQL, HTML/CSS/JS) et sur un hébergement gratuit (Alwaysdata) compatible avec la durée du projet
Temporel	Réalisation en environ 9 semaines de travail effectif, jalonnée en 5 sprints de 1 à 2 semaines chacun

1.3 Parties prenantes

La cartographie des parties prenantes distingue les acteurs internes du projet des acteurs externes qui influencent ou subissent ses résultats.

Parties prenantes internes

- **Développeur (Laurent MARC)** — chef de projet, développeur full-stack, rédacteur des livrables
- **Tuteur Studi** — accompagnement pédagogique, validation des étapes
- **Jury ECF** — évaluation finale de la certification

Parties prenantes externes (contexte fictif)

- **Sponsors du projet** — Julie et José, fondateurs fictifs du traiteur, qui incarnent le rôle de *Product Owner*
- **Utilisateurs finaux** — particuliers, entreprises, associations passant commande
- **Employés du traiteur** — équipe préparant et livrant les commandes
- **Hébergeur Alwaysdata** — fournisseur de l'infrastructure de production
- **Autorités de contrôle** — CNIL pour le RGPD, DGCCRF pour les CGV, Défenseur des droits pour l'accessibilité

Dans un contexte réel, la première étape consisterait à organiser un atelier de cadrage avec Julie et José pour affiner la vision produit, valider les priorisations et convenir d'un rythme de démonstration régulier. Dans le cadre pédagogique de l'ECF, ce dialogue est simulé par le développeur qui endosse alternativement les rôles de Product Owner et de développeur, en s'appuyant sur les grilles d'évaluation Studi comme voix du client.

2. Méthodologie retenue

Choix d'un modèle hybride mêlant prédictif et adaptatif, cadre Scrum et outils.

2.1 Choix d'un modèle hybride

Les modèles théoriques de gestion de projet se déclinent en quatre grandes catégories : prédictif, itératif, adaptatif et hybride. Chacun présente des avantages et des limites en fonction du contexte.

Modèle	Adapté quand	Limite dans notre cas
Prédictif (cascade, cycle en V)	Périmètre stable, exigences bien connues	Trop rigide face à l'apprentissage progressif du framework Symfony
Itératif / Incrémental	Projets complexes découpables en modules	Compatible avec le découpage en profils utilisateurs
Adaptatif (agile pur)	Environnement changeant, exigences incertaines	Difficile à appliquer seul sans équipe pour les cérémonies
Hybride	Contexte mixte, contraintes fortes en début et fin	Retenu pour ce projet

Le modèle finalement retenu est **hybride**, articulant trois phases distinctes :

Phase 1 — Cadrage prédictif

Définition du périmètre initial via un mini-cycle en cascade : recueil des besoins, cadrage MoSCoW, choix technologiques, modélisation MCD/MLD. Cette phase est nécessairement prédictive car les contraintes du référentiel ECF sont connues à l'avance et non négociables.

Phase 2 — Réalisation itérative et adaptative

Le développement suit un rythme Scrum simplifié avec des sprints de 1 à 2 semaines. Chaque sprint livre un incrément fonctionnel testable, et la priorisation du sprint suivant s'adapte aux enseignements du précédent (ajustements front-end après tests, refonte des formulaires admin après retours de validation).

Phase 3 — Finalisation prédictive

La dernière phase revient à un modèle prédictif : le périmètre est figé, le développement s'achève, la conformité légale est vérifiée point par point, la documentation est finalisée et le déploiement s'exécute selon une procédure documentée.

2.2 Cadre Scrum adapté

Le cadre Scrum classique repose sur trois piliers (transparence, inspection, adaptation) et trois rôles (Product Owner, Scrum Master, équipe de développement). Dans le contexte d'un projet ECF individuel, cette organisation doit être adaptée.

Rôles endossés par le développeur

Le développeur joue simultanément les trois rôles Scrum, ce qui nécessite une discipline particulière pour éviter les biais :

- **Product Owner** — priorise le backlog en s'appuyant sur les critères d'évaluation ECF et sur la vision produit fictive de Julie et José
- **Scrum Master** — garantit le respect des cadences, identifie les obstacles et adapte la méthodologie
- **Équipe de développement** — réalise les incréments techniques

Cérémonies simplifiées

- **Planification de sprint** — en début de chaque sprint, sélection des cartes Trello à traiter
- **Point quotidien allégé** — auto-check en début de journée sur les tâches en cours et les blocages
- **Revue de sprint** — en fin de sprint, démonstration mentale des fonctionnalités livrées face aux critères d'acceptation
- **Rétrospective** — auto-évaluation des points d'amélioration à reporter sur le sprint suivant

Definition of Done

Une carte Trello est considérée comme terminée lorsque :

- Le code fonctionne en local (tests manuels sur les parcours critiques)
- Le code fonctionne en production (test en HTTPS sur le site déployé)
- La documentation est à jour (README, doc technique si pertinent)
- Un commit est effectué sur la branche `develop` avec un message conventionnel

2.3 Outils de gestion

Le pilotage du projet s'appuie sur un ensemble d'outils choisis pour leur simplicité et leur gratuité.

Outil	Usage
Trello	Backlog produit, sprints, suivi des cartes (voir chapitre 5.1)
GitHub	Versioning du code source, workflow main/develop
VS Code	Environnement de développement, extension Symfony
Postman	Tests des 51 endpoints API pendant le développement
DBeaver	Administration de la base PostgreSQL en local et production
draw.io	Diagrammes MCD, MLD, UML pour la documentation technique
Microsoft Word	Rédaction des livrables PDF

Le choix de Trello comme outil principal de suivi de projet s'inscrit dans la philosophie Kanban : visualiser le flux de travail, limiter le nombre de tâches en cours, améliorer continuellement les processus. Trello combine avantageusement la simplicité du tableau Kanban avec la structuration en sprints Scrum.

3. Priorisation MoSCoW

Répartition des fonctionnalités selon leur priorité : Must, Should, Could, Won't.

La méthode MoSCoW permet de trier les fonctionnalités en quatre catégories mutuellement exclusives, chacune correspondant à un niveau d'engagement différent. Ce cadre a été utilisé dès le cadrage pour définir le périmètre livrable, puis appliqué comme filtre à chaque nouvelle demande ou idée émergente en cours de projet.

3.1 Must Have Fonctionnalités indispensables

Les Must Have constituent le socle sans lequel le projet ne peut pas être considéré comme livrable. Leur absence entraîne un échec de certification ou un site inutilisable.

Fonctionnalité	Justification
Système d'authentification 3 rôles	Base de tout le contrôle d'accès du site
Catalogue de menus avec filtres	Cœur métier de la vitrine commerciale
Passage de commande complet	Fonction principale attendue par un client
Espace client avec historique	Suivi indispensable après achat
Interface admin CRUD menus/plats	Autonomie de gestion du traiteur
Interface admin CRUD employés	Séparation des rôles obligatoire
Formulaire de contact	Attendu réglementaire (LCEN)
Mentions légales, CGV	Obligation légale française
Politique de mot de passe forte	Recommandation ANSSI
HTTPS avec certificat valide	Prérequis de sécurité en 2026
Déploiement en production	Site accessible au jury pour la soutenance

3.2 Should Have Fonctionnalités importantes

Les Should Have apportent une forte valeur ajoutée et sont attendues d'une plateforme professionnelle, mais leur absence n'empêche pas la validation du projet. Elles ont été traitées après les Must Have.

Fonctionnalité	Justification
Tableau kanban employé	Ergonomie fortement améliorée pour la gestion des commandes
Tableau de bord statistique admin	Aide au pilotage commercial
Intégration MongoDB (NoSQL)	Démonstration de compétence NoSQL
Fallback SQL automatique	Résilience architecturale
8 templates e-mail transactionnels	Communication client professionnelle
Système d'avis avec modération	Renforcement du lien client, preuve sociale
Rate limiting sur endpoints sensibles	Protection anti-abus (login, contact)
Timeline de suivi de commande	Transparence côté client
Déclaration d'accessibilité RGAA	Conformité EAA 2019/882

3.3 Could Have Bonus si le temps le permet

Les Could Have sont des fonctionnalités souhaitables mais non prioritaires. Elles sont abordées uniquement lorsque tous les Must et Should Have sont terminés.

Fonctionnalité	Statut final
Réduction commerciale automatique 10% > 500€	Livré
Prêt de matériel avec suivi de retour	Livré
Multi-thèmes de menus (Noël, Pâques, évènement, classique)	Livré
Gestion des horaires d'ouverture éditables	Livré
Filtres avancés par régime alimentaire	Livré
Menus similaires en fin de fiche produit	Livré

3.4 Won't Have Hors périmètre ECF

Les Won't Have sont explicitement exclus du périmètre courant, mais documentés comme évolutions futures. Leur identification en début de projet évite le *scope creep* et clarifie les attentes.

Fonctionnalité	Raison de l'exclusion
Paiement en ligne Stripe	Nécessiterait 2 semaines supplémentaires, prévu en phase 2
Application mobile native	Hors périmètre DWWM (compétence mobile native non enseignée)
Version anglaise	Non prioritaire pour un traiteur bordelais local
Multi-boutique / franchise	Hors périmètre commercial du contexte fictif
Programme de fidélité avec points	Idée intéressante mais non essentielle pour la V1
Chat en direct avec le traiteur	Remplacé par le formulaire de contact plus simple

Discipline MoSCoW : tout au long du projet, chaque nouvelle idée émergente a été confrontée à cette grille de priorisation. Les idées inspirantes mais chronophages ont été systématiquement reversées dans les Won't Have plutôt que d'être glissées dans le sprint en cours, ce qui a permis de tenir les délais.

4. Planification

Découpage du projet en cinq sprints, diagramme de GANTT et chemin critique.

4.1 Découpage en cinq sprints

Le projet est structuré en cinq sprints thématiques, chacun livrant un incrément fonctionnel cohérent. Cette structuration reprend la logique du *Work Breakdown Structure* en découpant par livrables métier plutôt que par tâches techniques.

Sprint 1 — Fondations

SEMAINES 1-2

Objectif : disposer d'un back-end opérationnel avec base de données et authentification

- Initialisation du projet Symfony 7 et configuration de l'environnement
- Modélisation MCD/MLD et création des 14 entités Doctrine
- Système d'authentification avec 3 rôles hiérarchiques et Argon2id
- Workflow de mot de passe oublié avec token hashé SHA-256
- Rate limiting sur les endpoints sensibles
- Setup Git avec branches main et develop

Sprint 2 — Catalogue

SEMAINES 3-4

Objectif : rendre visible le catalogue de menus côté client

- CRUD Menus et Plats côté back-end avec DTOs de validation
- Endpoints publics /api/menus avec filtres avancés
- Endpoints référentiels (thèmes, régimes, allergènes)
- Page publique du catalogue avec filtres réactifs (Alpine.js)
- Fiche détaillée d'un menu avec composition et sélecteur de personnes
- Page d'accueil avec hero et sections "Nos menus" et "Notre engagement"
- Formulaire de contact avec rate limiting anti-spam

Sprint 3 — Workflow de commande

SEMAINES 5-6

Objectif : permettre le parcours complet de commande côté client

- Création de commande avec verrou pessimiste sur le stock
- Page de commande avec récapitulatif dynamique et calcul de prix temps réel
- Espace client "Mes commandes" avec badges de statut
- Machine à états stricte pour les transitions de statuts
- Système d'avis avec modération et affichage RGPD-compliant
- Rédaction et intégration des 8 templates e-mail transactionnels

Sprint 4 — Administration

SEMAINES 7-8

Objectif : offrir des interfaces de pilotage complètes aux employés et à l'administrateur

- Interface kanban des commandes côté employé
- Modales de création et édition de plats avec allergènes multi-sélection
- Modales de création et édition de menus avec plats groupés par catégorie
- Tableau de bord administrateur avec KPI et graphiques
- Gestion des employés (création, activation, désactivation)
- Intégration MongoDB avec fallback SQL automatique
- Interface de modération des avis clients
- Gestion des horaires d'ouverture affichés publiquement

Sprint 5 — Finalisation

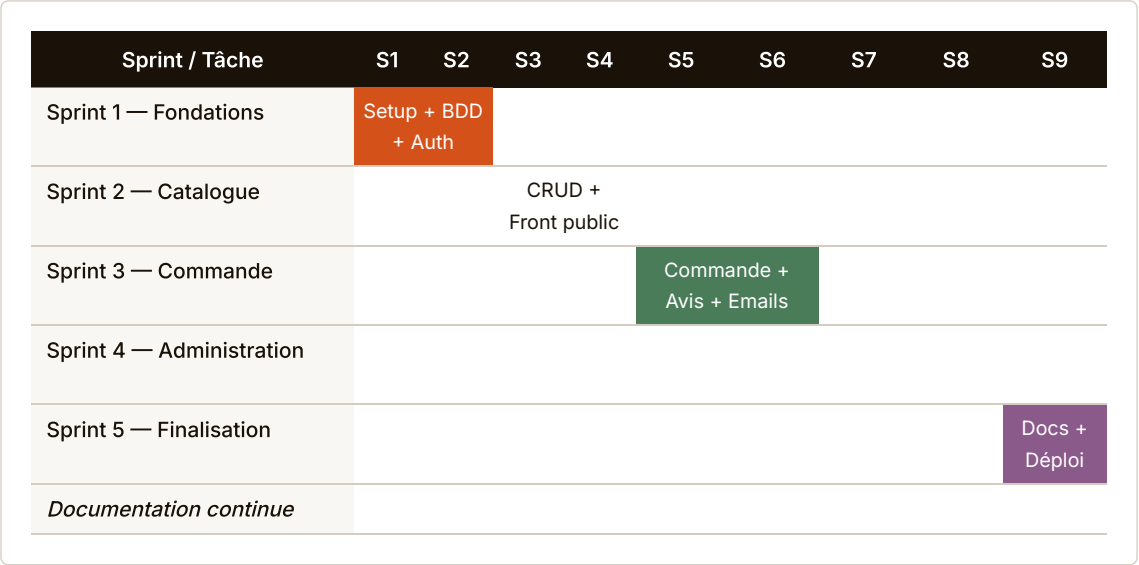
SEMAINE 9

Objectif : finaliser la conformité légale, la documentation et le déploiement

- Rédaction et intégration des pages Mentions légales, CGV, Déclaration d'accessibilité
- Audit sécurité et vérification des recommandations ANSSI
- Déploiement en production sur Alwaysdata avec HTTPS Let's Encrypt
- Rédaction des 5 documents livrables PDF
- Mise à jour du README GitHub
- Constitution du tableau Trello public
- Tests fonctionnels de bout en bout en production

4.2 Diagramme de GANTT

Le diagramme de GANTT ci-dessous représente l'enchaînement des sprints et met en évidence les dépendances entre livrables.



Le diagramme montre plusieurs points saillants du planning :

- Les sprints s'enchaînent séquentiellement mais présentent de légers chevauchements pour lisser la charge
- La documentation est produite en continu, pas seulement en fin de projet
- Le Sprint 5 est plus court (1 semaine) car la finalisation ne nécessite pas de nouveaux développements majeurs

4.3 Chemin critique

La méthode du chemin critique identifie l'enchaînement des tâches qui déterminent la durée totale du projet. Un retard sur une tâche du chemin critique entraîne un retard équivalent sur la livraison finale.

```
Debut → [MCD/MLD] → [Auth] → [CRUD Menus] → [Commande] → [Kanban] → [Deploiement] → [Docs] → Fin
   S1        S1         S1         S2             S3           S4         S5              S5

Duree du chemin critique : 9 semaines
```

Trois tâches concentrent l'essentiel du risque de retard :

- **Création commande avec verrou pessimiste (Sprint 3)** — logique métier complexe avec gestion de concurrence, difficile à tester exhaustivement
- **Refonte des formulaires admin (Sprint 4)** — la première version des formulaires plats/menus s'est révélée incompatible avec les DTOs et a nécessité une reprise complète
- **Déploiement en production (Sprint 5)** — première mise en ligne toujours porteuse

d'imprévus (permissions, variables d'environnement, extensions PHP)

Les tâches hors chemin critique disposent d'une marge de manœuvre qui a permis d'absorber les dépassements ponctuels sans compromettre la deadline finale.

5. Suivi opérationnel

Tableau Trello, rythme de travail et responsabilités via matrice RACI.

5.1 Tableau Trello

Le pilotage opérationnel du projet s'appuie sur un tableau Trello structuré en 8 listes reflétant le workflow Scrum/Kanban.

Structure des listes

1. 📋 **Backlog produit** — fonctionnalités identifiées lors du cadrage, avant affectation à un sprint
2. 🎯 **Sprint 1 — Fondations** — cartes du sprint 1
3. 🎯 **Sprint 2 — Catalogue** — cartes du sprint 2
4. 🎯 **Sprint 3 — Workflow commande** — cartes du sprint 3
5. 🎯 **Sprint 4 — Administration** — cartes du sprint 4
6. 🎯 **Sprint 5 — Finalisation** — cartes du sprint 5
7. ✅ **Terminé** — cartes complétées et validées
8. 🔮 **Évolutions futures** — Won't Have et idées post-ECF

Système d'étiquettes

Chaque carte porte au moins deux étiquettes : une pour sa priorité MoSCoW, une pour son domaine technique.

Étiquette	Couleur	Signification
Must Have	Rouge	Priorité MoSCoW indispensable
Should Have	Orange	Priorité MoSCoW importante
Could Have	Jaune	Priorité MoSCoW bonus
Won't Have	Violet	Hors périmètre, évolution future
Back-end	Bleu	Tâche PHP/Symfony/base de données
Front-end	Vert	Tâche HTML/CSS/JavaScript
Documentation	Gris	Rédaction de documents
DevOps	Marron	Déploiement, Git, hébergement

Format des cartes

Chaque carte suit un modèle unifié inspiré de Scrum :

- **Titre** avec émoji et libellé court
- **User story** au format « En tant que [rôle], je veux [action] afin de [objectif] »
- **Estimation** en points de complexité (Fibonacci : 1, 2, 3, 5, 8, 13)
- **Description détaillée** de la fonctionnalité
- **Checklist** de sous-tâches concrètes

Le tableau Trello complet est rendu public juste avant le rendu ECF et son lien figure dans le README GitHub du projet.

5.2 Rythme et cérémonies

Le rythme de travail respecte les principes agiles adaptés au contexte individuel.

Rythme quotidien

- **Démarrage** — revue rapide du Trello pour identifier la carte en cours
- **Session de travail** — plages de 1h30 à 2h avec technique Pomodoro
- **Point de bascule** — décision de continuer, changer de carte ou faire une pause
- **Clôture** — mise à jour du Trello et commit sur la branche `develop`

Rythme de sprint

- **Début de sprint** — sélection des cartes à partir du backlog en respectant l'ordre MoSCoW
- **Mi-sprint** — checkpoint sur l'avancement, ajustement du périmètre si nécessaire
- **Fin de sprint** — démonstration mentale des livrables, rétrospective des difficultés
- **Transition** — merge de `develop` vers `main` si le sprint a livré un incrément déployable

5.3 Matrice RACI

Bien que le projet soit conduit par un développeur unique, la matrice RACI clarifie les responsabilités entre les rôles fonctionnels endossés simultanément.

Activité	Développeur	Tuteur Studi	Jury ECF	Sponsor fictif
Définition du périmètre	R / A	C	I	C
Modélisation des données	R / A	C	I	I
Développement back-end	R / A	I	I	I
Développement front-end	R / A	I	I	I
Tests fonctionnels	R / A	I	I	I
Rédaction documentation	R / A	C	I	I
Déploiement production	R / A	I	I	I
Évaluation finale	I	C	R / A	I

Légende : R = Responsible (réalise) · **A = Accountable (garant)** · C = Consulted (consulté) · I = Informed (informé)

Cette matrice met en évidence que le développeur cumule la responsabilité de l'exécution (R) et de la garantie (A) sur la quasi-totalité des activités, ce qui est cohérent avec un projet individuel. Le tuteur intervient en conseil (C) sur les décisions structurantes, et le jury reste informé (I) jusqu'à l'évaluation finale où il prend la responsabilité de la certification.

6. Gestion des risques

Identification des risques principaux et plans de mitigation associés.

6.1 Identification des risques

L'analyse des risques a été menée en début de projet et actualisée à chaque fin de sprint. Sept risques principaux ont été identifiés, cotés selon leur probabilité et leur impact.

Risque	Probabilité	Impact	Criticité
Sous-estimation du temps de développement	Élevée	Élevé	Critique
Blocage sur une compétence technique nouvelle	Moyenne	Élevé	Important
Perte de données locale (poste développeur)	Faible	Élevé	Important
Incident hébergeur Alwaysdata avant soutenance	Faible	Élevé	Important
Scope creep (ajout continu de fonctionnalités)	Élevée	Moyen	Modéré
Faille de sécurité découverte tardivement	Moyenne	Moyen	Modéré
Documentation incomplète en fin de projet	Élevée	Moyen	Modéré

6.2 Plans de mitigation

Pour chaque risque identifié, un plan de mitigation a été défini et appliqué tout au long du projet.

Sous-estimation du temps de développement

- Découpage en petits sprints (1 à 2 semaines) pour ajuster fréquemment le cap
- Priorisation MoSCoW stricte pour reporter les fonctionnalités non essentielles
- Identification précoce des Won't Have pour éviter les distractions

Blocage sur une compétence technique nouvelle

- Recours à la documentation officielle Symfony et PostgreSQL en priorité
- Utilisation d'assistants IA pour explorer des solutions
- Développement de prototypes techniques isolés avant intégration

Perte de données locale

- Commit et push Git au minimum une fois par jour
- Dépôt GitHub distant comme sauvegarde canonique
- Documentation stockée dans un dossier de travail synchronisé

Incident hébergeur avant soutenance

- Déploiement finalisé plusieurs semaines avant la soutenance
- Sauvegarde manuelle de la base PostgreSQL avant chaque mise à jour
- Documentation de déploiement rigoureuse pour redéploiement rapide en cas de panne

Scope creep

- Toute nouvelle idée passe par le filtre MoSCoW avant intégration
- Les idées inspirantes mais chronophages sont versées dans les Won't Have
- Rétrospective de fin de sprint pour identifier les dérives

Faille de sécurité découverte tardivement

- Application des recommandations ANSSI dès le début (Argon2id, HTTPS, cookies HttpOnly)
- Rate limiting sur les endpoints sensibles dès le Sprint 1
- Audit sécurité complet en fin de Sprint 5 avant mise en production finale

Documentation incomplète en fin de projet

- Rédaction en continu tout au long du projet, pas seulement en fin de sprint 5
- README maintenu à jour à chaque nouvelle fonctionnalité majeure
- Documentation technique construite en parallèle du code, pas après

> **Bilan de la gestion des risques :** parmi les sept risques identifiés, cinq ont été maîtrisés avec succès. Deux se sont matérialisés en cours de projet — la sous-estimation du temps sur les formulaires admin (résolu par une refonte en fin de projet) et la documentation dense à rédiger en fin de parcours (compensée par un travail intensif sur la dernière semaine). Aucun risque critique n'a compromis la livraison finale.

7. Bilan et retour d'expérience

Ce qui a bien fonctionné, difficultés rencontrées, enseignements pour la suite.

7.1 Ce qui a bien fonctionné

La méthodologie hybride prédictif/adaptatif

Le choix d'un modèle hybride a été particulièrement pertinent. La phase prédictive initiale (cadrage MoSCoW, MCD/MLD) a permis de poser les fondations solides du projet. La phase adaptative Scrum a offert la souplesse nécessaire pour réviser certains choix en cours de route (refonte des formulaires admin, ajout du fallback SQL). La phase prédictive finale a garanti une clôture propre du projet dans les délais.

La priorisation MoSCoW disciplinée

Appliquer MoSCoW strictement à chaque nouvelle idée a évité les nombreuses dérives possibles. Le simple fait d'écrire une idée dans les Won't Have plutôt que de la traiter immédiatement a suffi à préserver le focus sur les Must Have.

Le workflow Git main/develop

La séparation entre `main` (production stable) et `develop` (intégration continue) a apporté une sécurité appréciable. Les expérimentations et refactorings ont pu être menés sur `develop` sans mettre en péril l'environnement de production, avec fusion vers `main` uniquement après validation.

Le Trello comme cockpit central

Concentrer toutes les tâches dans un unique tableau Trello a évité la dispersion. La visualisation en colonnes de sprint a rendu tangible l'avancement du projet, ce qui a soutenu la motivation dans les phases difficiles.

La documentation en continu

Rédiger la documentation au fil de l'eau plutôt qu'en fin de projet a permis d'éviter le mur documentaire de fin de parcours. La documentation technique s'est enrichie progressivement en même temps que le code, ce qui a garanti sa fidélité au produit livré.

7.2 Difficultés rencontrées

Refonte des formulaires administrateur

La première version des formulaires de création de menu et de plat côté admin s'appuyait sur des libellés textuels envoyés en snake_case, alors que les DTOs Symfony attendaient des identifiants numériques en camelCase. Cette incompatibilité a provoqué des erreurs 422 systématiques, invisibles côté utilisateur, et n'a été détectée qu'après plusieurs tentatives de création manuelle. La refonte complète des deux modales a nécessité environ une journée de travail en fin de projet.

Enseignement : aligner front et back sur le contrat d'interface (DTO) dès la conception, et tester chaque endpoint via Postman avant de construire l'interface graphique correspondante.

Configuration initiale d'Alwaysdata

Le déploiement initial a révélé plusieurs pièges de configuration : bundles Symfony désactivés en production, variable MAILER_DSN manquante, permissions insuffisantes sur `var/cache`, boucle de redirection sur les URLs propres. Chaque incident a nécessité entre 30 minutes et 2 heures de diagnostic.

Enseignement : tester le déploiement en environnement production dès le premier sprint, même sur un site vide, plutôt que d'attendre le sprint final. Documenter chaque incident au fil de l'eau (voir *Documentation de déploiement*).

Densité documentaire de fin de projet

Malgré la volonté de rédiger en continu, une part significative de la documentation a été produite dans les dernières semaines. Ce phénomène est probablement lié à la nécessité d'attendre que le produit soit stabilisé avant de le décrire précisément.

Enseignement : anticiper cette charge dans la planification et réserver explicitement les dernières semaines à la documentation plutôt qu'à de nouveaux développements.

Assumer les compromis architecturaux

Certains choix ont été des compromis assumés : MongoDB non actif en production au profit du fallback SQL, absence d'audit RGAA formel au profit d'une déclaration d'accessibilité honnête, absence d'environnement de staging au profit d'une procédure de rollback documentée. Chacun de ces compromis a demandé un travail argumentatif pour être défendu comme choix pertinent plutôt que comme lacune.

Enseignement : un compromis assumé et documenté vaut mieux qu'une prétention non tenue. La transparence sur les limites du projet renforce la crédibilité générale.

7.3 Enseignements pour la suite

Ce projet a permis d'identifier plusieurs enseignements transférables à de futurs projets professionnels.

Prévoir dès le cadrage les évolutions futures

Identifier les Won't Have en début de projet clarifie non seulement le périmètre courant mais dessine aussi une trajectoire d'évolution. Cette vision à moyen terme peut être communiquée aux sponsors pour montrer que les limites actuelles sont un choix conscient, pas un oubli.

Traiter la sécurité comme une exigence transverse

La sécurité ne peut pas être ajoutée en fin de projet. Elle doit être intégrée dès la modélisation (contraintes en base), la conception des DTOs (validation), le choix des mécanismes (Argon2id, HTTPS) et le déploiement (permissions, secrets). Un audit final ne peut que vérifier ces choix, pas les compenser.

Investir dans les outils de diagnostic

La maîtrise de Postman pour tester l'API, de DBeaver pour explorer la base, des logs Symfony pour comprendre les erreurs, s'est révélée déterminante pour maintenir un rythme soutenu. Investir du temps sur ces outils en début de projet paie sur toute la durée.

Automatiser dès que la manipulation devient répétitive

Les procédures de déploiement, sauvegarde et vidage de cache ont été rapidement automatisées via des scripts shell. Cette automatisation a réduit les erreurs humaines et documenté implicitement les procédures dans le code lui-même.

Documenter chaque incident au moment où il survient

Un incident vécu et résolu est une source d'apprentissage précieuse. Le noter immédiatement (dans une note, un fichier `NOTES.md`, ou directement dans la documentation officielle) évite d'avoir à reconstituer la mémoire des semaines plus tard.

Valoriser les compromis dans la communication

Un jury ou un sponsor n'attend pas la perfection, mais la maîtrise. Expliquer pourquoi telle fonctionnalité a été mise en Won't Have est aussi valorisant que présenter les fonctionnalités livrées. Cette lucidité sur ses choix reflète une maturité professionnelle.

> **Bilan général :** le projet Vite & Gourmand a atteint tous ses objectifs Must Have et Should Have, la quasi-totalité des Could Have, et a livré une plateforme fonctionnelle, sécurisée et documentée. Les difficultés rencontrées ont été surmontées et transformées en enseignements documentés. La méthodologie hybride, la priorisation MoSCoW disciplinée et l'usage du Trello ont constitué le socle du succès méthodologique.